

rheatflow

Benutzerhandbuch

**Echtzeit-Simulation thermischer Wärmenetze
für Lehre und Forschung**

Stand: Juli 2026

Hinweis zur Entstehung: Der Quellcode von rheatflow sowie dieses Handbuch wurden mit einem **KI-Coding-Agenten** (Claude Code, Anthropic) erstellt — unter menschlicher Anleitung: Anforderungen, fachliche Entscheidungen, Review und die Live-Verifikation gegen das laufende System stammen von Menschen. Die Wärmenetz-Physik ist gegen den **IBPSA-DESTEST-Cross-Tool-Benchmark** und gegen **reale Verbier-Messdaten** validiert (Kapitel 9). Quellcode und Handbuch stehen unter der MIT-Lizenz; die mitgelieferten Referenzdatensätze behalten ihre eigenen Lizenzen (IBPSA DESTEST: modifizierte BSD-3; pandapipes-Beispielnetze: BSD-3; OpenDHN/Verbier: CC BY 4.0). Projektseite: <https://github.com/markisbell/rheatflow>

Inhaltsverzeichnis

1. Einführung	5
1.1. Was ist rtheatflow?	5
1.2. Die drei Sichten	5
1.3. Die drei Anwendungen	6
1.4. Das Zeitmodell	6
1.5. Validierte Physik	7
1.6. Weiterführende Dokumentation	7
2. Installation und Start	9
2.1. Windows (Entwicklung, ein Klick)	9
2.2. Lokaler Start (Entwicklung, manuell)	9
2.3. Docker Compose (Gesamtstack)	10
2.4. Kaltstart: der numba-JIT-Aufschlag	10
3. Arbeitsablauf: Netz – Lasten – Live	11
3.1. NetzStudio: die drei Spalten	11
3.2. Die Live-Ansicht und die Steuerleiste	13
4. Die Karte und die Farbebenen	14
4.1. Die vier Farbebenen	14
4.2. Unbekanntes ist unbekannt	15
4.3. Popups und Element-Kontextmenü	15
5. Beobachtbarkeit: Messen und Schätzen	16
5.1. Messgeräte platzieren	16
5.2. Strikter Modus	18
5.3. Der Vorwärts-Beobachter (die Schätzung)	18
5.4. Ehrlichkeits-Stolperdrähte	20
5.5. Der Blindflug der Schlechtpunktregelung	20
6. Physik und Regelungen	22
6.1. Das Vor-/Rücklaufmodell	22
6.2. Die bidirektionale Lösung und die Retry-Ladder	22
6.3. Abgeleitete Größen: Verluste und Einspeisung	23
6.4. Die Heizkurve	24
6.5. Die Schlechtpunktregelung	25

7. Anlagen	26
7.1. Erzeuger	26
7.2. Pufferspeicher	27
7.3. Einspeiser und Bypass	27
7.4. Die Nullfluss-Schwelle	28
8. Der Wetter-Regler	29
8.1. Die Gradstunden-Skalierung	29
9. Referenznetze und validierte Physik	31
9.1. Der Katalog	31
9.2. DESTEST — der validierte Cross-Tool-Benchmark	31
9.3. Schutterwald — Plausibilität am realen Ort	33
9.4. Verbier — gegen reale Messdaten	33
9.5. Akzeptierte Modellunterschiede	34
10. Szenarien	35
11. Daten exportieren	36
11.1. Aufzeichnung	36
11.2. Tage-Export (Bulk)	36
11.3. Der experimentelle Transient-Modus	37
12. Langzeit-Visualisierung mit Grafana	38
13. Konfiguration	39
A. REST-/WebSocket-Schnittstelle	41
A.1. Das StepResult-Wireformat	42
B. Eingabedateiformat	43
C. Fehlerbehebung	45

1. Einführung

1.1. Was ist rheatflow?

rheatflow ist eine Lehr- und Forschungsplattform für thermische *Wärmenetze* (Fernwärme). Sie löst fortlaufend die Thermo-Hydraulik eines Zweileiter-Fernwärmenetzes auf Basis von [pandapipes 0.14.0](#) und stellt die Ergebnisse live im Browser dar — auf einer echten OpenStreetMap-Karte, in der sich Vorlauf- und Rücklauftemperaturen, Massenströme, Drücke und Wärmeverluste Minute für Minute entwickeln. Anlagen (Erzeuger, Pufferspeicher, Einspeiser, Bypässe, Abnehmer) lassen sich im laufenden Betrieb platzieren; eine Heizkurve und eine Schlechtpunktregelung schließen die Betreiberschleife.

rheatflow ist das *Wärme-Schwesterprojekt* des Stromnetzsimulators [rtpowerflow](#) (netzsim) und spiegelt dessen Architektur — Elektrizität wird konsequent auf Wärme übersetzt: Spannung wird zu Temperatur, Strom zu Massenstrom, Wirkleistung zu Wärmestrom, die Leitung zur Trasse, der Trafo zur Heizzentrale.

Das didaktische Kernkonzept ist die Trennung von **drei Sichten** auf dasselbe Netz.

1.2. Die drei Sichten

Realität Der vollständige, physikalisch gelöste Zustand jeder Trasse, jedes Knotens, jedes Abnehmers — das, was im Netz *tatsächlich* passiert. In der Wirklichkeit kennt ihn niemand.

Gemessen Nur das, was platzierte Messgeräte (Wärmemengenzähler, T/p-Sensoren) tatsächlich erfassen — und nur in dem Zeitraster, das der gewählte Messmodus liefert. Alles andere ist grau und unbekannt („unbekannt ist unbekannt“). Das ist der Standardblick des Betreibers.

Schätzung Das, was ein Betreiber aus seinen wenigen Messwerten *berechnen* kann: ein zweites Netzmodell, angetrieben allein von Messwerten und Erwartungsprofilen — inklusive des Fehlers, den diese Rechnung gegenüber der Wahrheit macht.

Der intellektuelle Anspruch, den die Plattform im Code verteidigt: *Die Sichten Gemessen und Schätzung dürfen ausschließlich Informationen enthalten, die ein realer Betreiber tatsächlich hätte*. Keine Ebene oberhalb der Realität darf um die Projektion herum an die Wahrheit greifen — und das ist kein Vorsatz, den man sich merken muss, sondern

durch Tests (die „Ehrlichkeits-Stolperdrähte“, Abschnitt 5.4) und zur Laufzeit durch den strikten Modus (Abschnitt 5.2) erzwungen.

Typische Lehrfragen, die sich damit beantworten lassen

- Was passiert mit dem Schlechtpunkt- Δp , wenn morgens alle Übergabestationen gleichzeitig aufheizen?
 - Wie ändert sich die Verlustquote, wenn man von einem 3G-Netz (110/60) auf ein 4G-Netz (70/40) umschaltet — und was kostet das an Pumpenstrom?
 - Wie viele Wärmemengenzähler braucht die Zustandsschätzung, um eine Rücklauf-Anomalie an einer einzelnen Station überhaupt zu *sehen*?
 - Kappt ein 100-kWh-Pufferspeicher die morgendliche Wärmespitze wirksam — und wann hört er von selbst auf zu laden?
-

1.3. Die drei Anwendungen

Die Plattform besteht aus drei Anwendungen, die gemeinsam über eine einzige `docker-compose.yml` starten:

1. **rtheatflow** — der Simulationskern: eine FastAPI-Anwendung (`src/`), die ein Netz samt Tagesprofilen lädt und fortlaufend die Thermo-Hydraulik löst; jedes Ergebnis ist per REST abrufbar und wird zusätzlich über einen WebSocket an alle verbundenen Browser gepusht (Backend, **Port 8001**).
2. **Web-Oberfläche** — die React-Anwendung im Browser (`ui/`, deutschsprachig, oben rechts auf EN umschaltbar), über die alle Funktionen dieses Handbuchs bedient werden (Vite-Entwicklungsserver **Port 5174**, in Docker per nginx auf **Port 8081**).
3. **Visualisierungs-Stack** — ein Collector holt jeden gelösten Schritt per REST ab und schreibt ihn nach InfluxDB; ein vorkonfiguriertes Grafana-Dashboard zeigt die Langzeitverläufe (Kapitel 12, Grafana **Port 3001**, InfluxDB **Port 8087**).

Das Geschwister-Portschema. Die Ports 8001/5174 (und 8081/8087/3001 im Docker-Stack) sind *mit Absicht* gewählt: netzsim/rtpowerflow belegt 8000/5173 (bzw. 8080/8086/3000). So laufen beide Plattformen — Strom und Wärme — parallel auf derselben Maschine, ohne sich die Ports streitig zu machen.

1.4. Das Zeitmodell

Ein Simulationstag besteht aus **1440 Schritten zu je einer Minute**. Für jeden Schritt werden Lasten, Wetter, Heizkurve und Anlagen auf die Werte des Zeitschritts gesetzt und ein vollständiger, eingeschwungener Thermo-Hydraulik-Zustand gelöst (auf üblichen

Rechnern warm etwa 20–35 ms, siehe Kapitel 9). Die Simulation läuft im *beschleunigten Takt*: pro **Schrittdauer** realer Sekunden schreitet die Simulationsuhr eine Minute voran.

Schrittdauer	Beschleunigung	ein Simulationstag dauert
1,0 s (Vorgabe)	60×	24 Minuten
0,5 s	120×	12 Minuten
0,1 s	600×	2,4 Minuten

Nach Schritt 1439 (23:59 Uhr) springt die Simulation auf Schritt 0 zurück, zählt den Tag hoch und wiederholt die Tagesprofile. Schrittindex und Uhrzeit rechnen sich direkt ineinander um: $\text{Schritt} = h \cdot 60 + \text{min}$.

Beispiel 07:20 Uhr entspricht Schritt $7 \cdot 60 + 20 = 440$. Wer per API an die morgendliche Aufheizspitze springen will, ruft daher `POST /control/seek` mit `{step: 440}` auf — die Oberfläche erledigt dasselbe über den Zeit-Regler in der Steuerleiste.

1.5. Validierte Physik

Ein Lehrwerkzeug ist nur so glaubwürdig wie seine Physik. Die Thermo-Hydraulik von rheatflow ist deshalb gegen *externe* Wahrheit validiert, nicht gegen frühere eigene Ausgaben: gegen den **IBPSA-DESTEST**-Cross-Tool-Benchmark (mehrere unabhängige Gebäudesimulations-Werkzeuge rechnen dasselbe kleine Netz und veröffentlichen ihre Ergebnisse — die Streuung zwischen den Werkzeugen ist selbst die Toleranz) und gegen **reale Messdaten** des vermaschten Alpennetzes Verbier (150 Übergabestationen). Als Plausibilitätsprüfung dient zusätzlich das reale Ortsnetz Schutterwald. Die vollständigen Zahlen, die akzeptierten Modellunterschiede und die Reproduktionskommandos stehen in Kapitel 9. Kurzfassung: Die Vorlauftemperaturen der 150 Verbier-Stationen werden im Median auf **0,83 K** genau getroffen, ohne dass diese Temperaturen jemals Eingabe wären.

1.6. Weiterführende Dokumentation

Dieses Handbuch ist die bedienungsorientierte Einführung. Für die Tiefe gibt es im Verzeichnis `docs/` fünf technische Deep-Dives (englisch, gegen den Code mit `Datei:Zeile`-Ankern geschrieben): **ARCHITECTURE.md** (System, Wireformat, Solverpolitik, Beobachtbarkeit, quasistatische Ehrlichkeit), **PHYSICS_AND_CONTROLS.md** (pandapipes-Modell, Retry-Ladder, Verluste, Heizkurve, Schlechtpunkt, Anlagen), **OBSERVABILITY.md** (die drei Ebenen, der Vorwärts-Beobachter, die Stolperdrähte), **REFERENCE_NETWORKS.md** (der Fünf-Dateien-Vertrag, alle Netze) und **BENCHMARKS.md** (Rechenzeiten + Validierungsprotokoll).

`SPEC.md` ist die bindende Spezifikation, `docs/API.md` die generierte Endpunktreferenz. Das vorliegende Handbuch wird zusätzlich unter `GET /manual` direkt aus dem Repository als HTML ausgeliefert.

2. Installation und Start

2.1. Windows (Entwicklung, ein Klick)

Der einfachste Weg auf einem Windows-Rechner:

```
start_rtheatflow.bat
```

Der Starter erkennt bereits laufende Server (und startet sie nicht doppelt), startet Backend (Port 8001) und Vite-UI (Port 5174) je in einer eigenen Konsole, wartet auf `/health` und öffnet den Browser. Der *allererste* Rechenschritt kompiliert die numba-Kerne vor (Kaltstart, bis ≈ 60 s, Abschnitt 2.4); solange zeigt der Starter „Warte auf das Backend“. **Beenden:** `stop_rtheatflow.bat` schließt beide Serverfenster samt eventuell verwaister Hintergrundprozesse dieses Repos — ein parallel laufendes `netzsims/rtpowerflow` (Ports 8000/5173) bleibt unberührt.

Einmalige Vorbereitung (die virtuelle Umgebung anlegen):

```
py -3 -m venv .venv
.venv\Scripts\pip install -r requirements.txt
```

`ui\node_modules` wird beim ersten Start automatisch per `npm install` nachgezogen.

2.2. Lokaler Start (Entwicklung, manuell)

Wer die beiden Prozesse selbst in der Hand haben will, startet sie getrennt:

```
:: Backend (Port 8001)
set PYTHONPATH=src
.venv\Scripts\python -m rtheatflow.main

:: UI (Port 5174; leitet /api und /ws an 127.0.0.1:8001 weiter)
cd ui && npm run dev
```

Der Vite-Server bindet bewusst `127.0.0.1` (nicht `localhost`) als Proxy-Ziel und läuft mit `strictPort` — er scheitert lieber laut, als still auf einen anderen Port auszuweichen und dann ins falsche Backend zu proxen (Abschnitt C erklärt die Windows-IPv6-Falle dahinter).

2.3. Docker Compose (Gesamtstack)

Der komplette Stack — Backend, UI, InfluxDB, Collector und Grafana — startet mit einem Befehl:

```
docker compose up --build
```

Dienst	Adresse	Inhalt
ui	http://localhost:8081	Karten-Oberfläche (nginx)
backend	http://localhost:8001	REST + WebSocket; Swagger unter <code>/docs</code> , Monitor unter <code>/</code> , Handbuch unter <code>/manual</code>
grafana	http://localhost:3001	Fernwärme-Dashboard (admin / admin)
influxdb	http://localhost:8087	Zeitreihenspeicher (Entwicklungs-Zugangsdaten)

Das Backend-Image bäckt das mitgelieferte `data/`-Verzeichnis ein und läuft eigenständig; das Volume `./data` macht Aufzeichnungen und importierte Netze dauerhaft. Alle Zugangsdaten im Compose-File sind **Entwicklungs-Vorgaben** (Lehrwerkzeug, bewusst ohne Authentifizierung) — vor einem Einsatz außerhalb einer vertrauenswürdigen Maschine ändern.

2.4. Kaltstart: der numba-JIT-Aufschlag

pandapipes verwendet numba mit *bloßem* `@jit` (ohne `cache=True`), also ohne Cache-Datei: der erste Lastfluss eines jeden Prozesses kompiliert die Pipeflow-Kerne frisch — etwa 7 s reine JIT-Zeit, dazu kommt der Import von `pandapipes/numba/llvmlite` selbst (4,9–12,1 s, stark schwankend). Das Backend versteckt das: unter `AUTOSTART` läuft der erste Rechenschritt in einem Hintergrund-Thread, sodass die JIT-Kompilierung das Binden des Sockets nicht blockiert. Praktisch heißt das: Der *erste* veröffentlichte Frame trägt eine `solve_ms` in den Tausenden — das ist JIT, kein langsames Netz. Jeder Folgeschritt ist warm.

3. Arbeitsablauf: Netz – Lasten – Live

Die Oberfläche ist wie ein klassisches Desktop-Programm organisiert: Beim Start öffnet sich die *Live-Ansicht*, die Menüleiste am oberen Rand bündelt alle Funktionen. Ganz oben in der Leiste:

Datei alles, was lädt, speichert oder exportiert: Szenarien speichern/laden/löschen (Kapitel 10) sowie Aufzeichnung und Tage-Export (Kapitel 11). Läuft eine Aufzeichnung oder ein Export, erscheinen rechts in der Leiste klickbare Status-Chips (**REC n / Export p %**).

Ansicht wählt die Farbebene der Karte (Vorlauf, Rücklauf, Geschwindigkeit, Differenzdruck — Kapitel 4).

Hilfe öffnet dieses Handbuch (/manual), die Swagger-Doku (/docs) und die Projektseite.

Rechts neben den Menüs sitzen zwei dauerhaft sichtbare Umschalter: der **Bereich Live / NetzStudio** (zwischen laufender Simulation und Netz-Konfiguration wechseln) und der **Sicht-Umschalter Realität / Gemessen / Schätzung** (Kapitel 5) — die drei Sichten sind das Kernkonzept, deshalb ist die aktive Sicht immer ablesbar und mit einem Klick wechselbar. Ganz rechts steht der Netz-Chip (Name · Abnehmerzahl · Trassenkilometer) und der DE/EN-Umschalter.

Der typische Arbeitsablauf ist ein Dreischritt — Netz wählen, Lasten erzeugen, live beobachten. Die ersten beiden Schritte leben im Bereich **NetzStudio**, der in drei Spalten von links nach rechts durchlaufen wird.

3.1. NetzStudio: die drei Spalten

Spalte 1 — Netz wählen oder importieren. Die linke Spalte listet alle verfügbaren Netze, gruppiert in **Bibliothek** (die mitgelieferten Katalognetze) und **Eigene Netze** (per POST /networks/import importierte Fünf-Dateien-Bündel). Ein **Netz importieren. . .** -Dateidialog nimmt entweder die fünf Vertragsdateien zusammen oder ein einzelnes Bündel entgegen, prüft sie und legt das Netz als **source=üser** ab. Der Katalog enthält zwei Lehrnetze (**Demo-Dorf, Appendix A**) und die **Referenznetze** DESTEST (8/16/32 Gebäude), Schutterwald und Verbier — alle mit Herkunft und validierten Zahlen (Kapitel 9).

Spalte 2 — Lastpolitik. Die Mittelspalte weist den Abnehmerknoten realistische Tagesprofile zu (die *Lastpolitik*, SPEC §4.5). Grundlage ist ein mitgelieferter Archetyp-Katalog: physikbasierte Jahresprofile aus **demandlib** (VDI 4655, Raumheizung) und **OpenDHW** (stochastisches Warmwasser). Drei Archetypen sind ausgeliefert:

Archetyp	Beschreibung	Jahres-Raumwärme
EFH_ALT_4P	unsaniertes Einfamilienhaus, 4 Pers.	18 MWh/a
EFH_SAN_4P	KfW-saniertes EFH, 4 Pers.	9 MWh/a
MFH_ALT_10WE	Mehrfamilienhaus, 10 Wohneinheiten	95 MWh/a

Einstellbar sind: der Gebäudemix (Archetyp-Auswahl), der Zuweisungsmodus (`round_robin` / `random`), ein **Seed**, eine **Skalierung**, das **Tag-Perzentil** (0,9 = kalter Wintertag, 0,5 = Übergang, 0,05 = Sommer), der **Jitter** (deterministische ± 30 -min-Verschiebung und 0,92–1,08-Amplitudenstreuung, damit gleiche Gebäude nicht synchron ziehen), die DHW-Varianten und das Temperaturniveau **3G/4G**. Das Temperaturniveau koppelt sowohl die Archetyp-Rücklauftemperaturen als auch die Heizkurve (3G-Rückläufe sind wärmer als 4G, siehe Tabelle 3.1).

Rücklauftemperatur	3G-Preset	4G-Preset
EFH_ALT_4P	55 °C	45 °C
EFH_SAN_4P	45 °C	35 °C
MFH_ALT_10WE	50 °C	40 °C

Tabelle 3.1.: Die vom Temperaturniveau gekoppelten Archetyp-Rücklauftemperaturen (`TRETURN_C` in `loadgen/`).

Spalte 3 — Prüfen & Anwenden. Die rechte Spalte zeigt die Vorschau: eine Zuweisungstabelle, Kennzahlen (Auslegungslast, Spitze, Jahresenergie, Trassenlänge) und die **lineare Wärmedichte** [MWh/(m·a)] als eingefärbte Kachel gegen die Faustregel ≥ 1 –1,5, dazu eine Jahresdauerlinie als Sparkline gegen die Auslegungslast. **Anwenden** lädt das Netz in die Engine (der Live-Bereich öffnet sich neu, die Uhr läuft ab Tag 0 / Schritt 0). Nichts an der Vorschau verändert die laufende Simulation — erst **Anwenden** wirkt.

Warum die lineare Wärmedichte über die Wirtschaftlichkeit entscheidet Wählt man das **Demo-Dorf**, meldet die KPI-Kachel eine lineare Wärmedichte von etwa 0,39–0,46 MWh/(m·a) — *unter* der Faustregel ≥ 1 , die Kachel färbt sich warnend. Das ist kein Fehler, sondern der Lehrpunkt: Ein dünn besiedeltes Straßendorf mit langen Trassen und wenig Anschlussdichte ist für ein klassisches Fernwärmenetz kaum wirtschaftlich — die stehenden Verluste der Trasse verteilen sich auf zu wenig verkaufte Wärme. Das reale Ortsnetz Schutterwald erreicht mit seinem 50-m-Anschlussradius dagegen 1,34 MWh/(m·a), genau im tragfähigen Band.

3.2. Die Live-Ansicht und die Steuerleiste

Nach **Anwenden** zeigt der Bereich **Live** die Karte (links, Kapitel 4) und eine in der Breite ziehbare Seitenleiste (rechts) mit einklappbaren Abschnitten: **Übersicht**, **Schlecht-punkt**, **Messstellen**, **Heizkurve**, **Wetter** und den per Strg-Klick angehefteten Element-Abschnitten. Am unteren Rand liegt die **Steuerleiste**:

Start/Pause hält die Simulationsuhr an bzw. lässt sie weiterlaufen.

Zeit-Regler springt an eine Tageszeit (Schritt 0...1439).

Tag-Regler erscheint nur bei mehrtägigen Wetterhorizonten (`n_days>1`).

Schrittdauer 0,1–1 s echte Zeit pro Simulationsminute.

Jeder Steuerbefehl synchronisiert die Anzeige aus der Antwort der Engine; die Uhr selbst wird vom WebSocket getrieben und liegt auch bei 10 Schritten/s höchstens einen Frame hinter dem Backend.

4. Die Karte und die Farbebenen

Die Karte ist eine rohe Leaflet-Karte (ohne react-leaflet) auf CARTO/OSM-Kacheln. Sie ist *einmal pro Topologie* aufgebaut — eine Polylinie je Trasse, ein Kreismarker je Abnehmer, Marker für Erzeuger, Anlagen und Sensoren — und wird pro Frame nur *umgefärbt* (`.setStyle()`), nie neu gebaut. So bleibt die Darstellung auch bei hohem Tempo flüssig.

4.1. Die vier Farbebenen

Das Menü **Ansicht** (oder der Umschalter auf der Karte) wählt, welche Größe die Trassen und Marker einfärbt. Die Rampen sind *domänenverankert* — nicht auf das Min/Max des Augenblicks normiert, sondern an physikalisch bedeutsamen Schwellen fixiert, sodass eine Farbe immer dasselbe bedeutet:

Vorlauftemperatur (Standard) warme Rampe (Gelb→Rot); „ganz heiß“ liegt *exakt* bei der Auslegungs-Vorlauftemperatur der aktiven Heizkurve. Ein 4G-Netz mit 70°C glüht bei 70 genau so wie ein 3G-Netz bei 110 — die Rampe folgt dem Auslegungspunkt.

Rücklauftemperatur kühle Rampe (Blau→Violett) über 25–70°C. Hohe Rückläufe (schlechte Auskühlung, das „Low- ΔT -Syndrom“) springen violett heraus und fallen sofort auf.

Geschwindigkeit Grün (Ruhe) → Bernstein genau ab **1,5 m/s** → Rot ab **3 m/s** (Kapazitätsgrenze); das Vorzeichen wird ignoriert, weil Rücklaufrohre „rückwärts“ fließen.

Differenzdruck Ampel an den Abnehmermarkern: Rot streng *unterhalb* des Mindest- Δp (DP_MIN_BAR, Vorgabe **0,5 bar**), Bernstein in einem 0,15-bar-Rand darüber, sonst Grün; die Trassen treten dabei zurück.

Jede Ebene bringt ihre eigene Farbleiste (Legende) mit, deren oberste Marke bei der Vorlaufe Ebene die live gelesene Auslegungstemperatur zeigt.

4.2. Unbekanntes ist unbekannt

Das Leitprinzip der Farbgebung: *Was nicht gemessen ist, wird als unbekannt dargestellt* — niemals in einer „gesunden“ Farbe. Ein Element ohne Messwert bekommt ein eigenes, gedämpftes Grau (UNOBSERVED #39424f für Marker, UNOBSERVED_LINE #2b323c für Trassen) plus ein 6 6-Strichmuster, das keine gesunde Rampe je erzeugen kann. In der Sicht **Gemessen** tragen die Trassen gar keine Zähler — sie sind deshalb grundsätzlich grau-gestrichelt; die Abnehmer werden nur aus `measurements.consumers` gefärbt. Ein Standard-Zähler, der noch in seinem ersten 15-Minuten-Fenster steckt, erscheint mit gestricheltem 3 3-Ring und gedämpfter Füllung (ehrlicher Kaltstart, Kapitel 5).

4.3. Popups und Element-Kontextmenü

Ein einfacher Klick auf eine Trasse, einen Abnehmer oder Erzeuger öffnet ein Popup mit den Live-Werten. Popups sind *lebendig*: Der Inhalt liest bei jedem Frame den neuesten Zustand, ein offenes Popup aktualisiert sich also laufend — in der Realitätssicht tickt „Vorlauf → Rücklauf“, in der Sicht Gemessen steht dort „keine Messung — unbekannt“, ganz ohne Neubinden.

Strg-Klick heftet ein Element als eigenen Abschnitt in die Seitenleiste (mit Live-Werten und Konfigurationsreglern; ein Pufferspeicher zeigt dort seinen SoC-Balken). **Rechtsklick** öffnet das Element-Kontextmenü — die Grammatik zum Platzieren und Konfigurieren von Anlagen und Messgeräten (Kapitel 7 und 5).

5. Beobachtbarkeit: Messen und Schätzen

Dies ist das Herzstück der Plattform. Ein realer Fernwärmebetreiber sieht nie den wahren Zustand seines Netzes. Er sieht eine Handvoll Wärmemengenzähler und Druck-/Temperatursensoren sowie die Instrumentierung der Heizzentrale — und aus diesem spärlichen Bild muss er entscheiden, wie hart die Pumpe läuft und wie heiß der Vorlauf ist. In der Lücke zwischen wahren und gemessenem Zustand leben die Betriebsfehler: ein ausgehungerner Abnehmer am Netzende (der *Schlechtpunkt*), den kein Sensor abdeckt; ein hoher Rücklauf, der still Pumpenstrom verschwendet; eine Störung, die keinen Alarm auslöst, weil kein Instrument sie erfasst. *rheatflow* macht diese Lücke sichtbar und *erzwingt* sie.

5.1. Messgeräte platzieren

Drei Gerätearten stehen zur Verfügung. Zwei sind platzierbar (Rechtsklick auf das Element → Kontextmenü, verwaltet im Seitenpanel-Abschnitt **Messstellen**); die dritte ist immer da:

Wärmemengenzähler an einer Übergabestation (Abnehmer): liefert Leistung, Massenstrom, Vor- und Rücklauftemperatur sowie Δp (Kanäle `q_kw`, `mdot_kg_per_s`, `t_supply_c`, `t_return_c`, `dp_bar`).

T/p-Sensor an einem Trassenknoten: Druck und Temperatur in Vor- und Rücklauf des Knoten-Paares.

Erzeuger-SCADA an der Heizzentrale, **immer** vorhanden — echte Heizwerke messen sich selbst. Sie ist Telemetrie, keine Messstelle, erscheint nie als Platzierung und degradiert nie in das 15-Minuten-Raster.

Trassen tragen bewusst *keine* Messung — Leitungsströme und Rohrverluste bleiben unbekannt, solange die Wahrheit ausgeblendet ist. Gerade das höchstbelastete Rohr ist also unsichtbar, wenn dort niemand misst.

Die Summe lügt nicht — sie ist nur unvollständig Die Sicht **Gemessen** zeigt in der Übersicht die „gemessene Nachfrage (Σ)“: die Summe nur der Zähler, die *gerade einen Wert liefern*. Ein unbemessener Abnehmer trägt nichts bei, ein Standard-Zähler im Kaltstart-Fenster trägt nichts bei. Diese Zahl ist also ehrlich *kleiner* als die wahre Nachfrage, sobald die Abdeckung lückenhaft ist. Die SCADA-Kennzahlen der Zentrale

(Einspeisung, Vorlauf, Rücklauf, Pumpenstrom) sind dagegen belastbar, weil durch die Zentrale *alles* fließt. Diese Asymmetrie — Summenmessung an der Zentrale belastbar, Ortsauflösung im Netz null — ist der Kern des Beobachtbarkeitsproblems.

Vorlagen (Presets)

Der Abschnitt **Messstellen** bietet vier Vorlagen, die die Platzierung jeweils *komplett ersetzen* (Sammelaktionen, keine additiven Änderungen):

Alle Abnehmer ein Zähler an jeder Übergabestation — volle Abdeckung, die Vorgabe.

Nur Erzeuger nur das T/p-Paar an der Zentrale (SCADA ist ohnehin immer an).

Schlüsselstellen Zentrale + T/p an den Netzen + ein Zähler am *aktuell bekannten* Schlechtpunkt (aus dem letzten konvergierten Frame; vor dem ersten Rechenschritt ist keiner bekannt und keiner wird gesetzt — ehrlich).

Alles entfernen keine Geräte — Blindflug auf reiner Erzeuger-SCADA.

Zähler-Modus: voll oder 15 Minuten

Ein einziger Modus-Schalter (POST /measurements/mode) gilt für alle platzierten Geräte:

Voll jeder Kanal, jeder Simulationsschritt.

Standard (15 min) 15-Minuten-Fenstermittelwerte, ausgerichtet an der *Simulationszeit* (Fenster $w = \text{Tick} // \text{Fensterschritte}$, tagesübergreifend stabil) — das Verhalten eines deutschen Lastgangzählers. Der Kanal liest den Mittelwert des zuletzt *abgeschlossenen* Fensters und liefert bis zur ersten Fenstergrenze nach dem Platzieren ehrlich **gar nichts** (null).

Der **ehrliche Kaltstart** ist die Pointe: Ein frisch gesetzter Standard-Zähler zeigt **null** — nicht null kW, nicht den Momentanwert — bis sein erstes 15-Minuten-Fenster schließt. Ein Umschalten des Modus setzt den Fensterzustand zurück (nie ein gefälschter Momentanwert aus einem alten Akkumulator), ein entferntes Gerät verliert seinen Fensterzustand, ein neu gesetztes startet wieder kalt.

Warum das 15-Minuten-Raster Spitzen verschluckt Zieht ein Abnehmer zwischen 07:02 und 07:12 Uhr zehn Minuten lang eine Warmwasser-Spitze, zeigt ein **Voll**-Zähler diese Spitze minutengenau. Ein **Standard**-Zähler liefert für das Fenster 07:00–07:15 nur den Mittelwert — rund $\frac{10}{15}$ der Spitze, ein Drittel ist *in den Messdaten* verschwunden (im Rohr floss es natürlich). Der Modus-Schalter macht den Unterschied unmittelbar sichtbar — und mit ihm die Frage, ob die Schlechtpunktregelung dieser Spitze überhaupt folgen kann (Abschnitt 5.5).

5.2. Strikter Modus

Ist der Server im strikten Modus konfiguriert (`RTHEATFLOW_EXPOSE_GROUND_TRUTH=false`), verlässt die Wahrheit den Server gar nicht erst. Der Strip passiert an genau *einer* Stelle (`StateStore._project`): Die vier Wahrheits-Schlüssel (`junctions`, `pipes`, `consumers`, `summary`) werden entfernt und das Freitext-Feld `error` geleert. `/state`, `/history`, jeder WebSocket-Frame und (seit der Aufzeichnung) auch die CSV-Pakete gehen durch diesen einen Pfad — der strikte Modus lässt sich also nicht durch einen vergessenen Endpunkt umgehen. Der Sicht-Umschalter fällt in der Oberfläche automatisch von **Realität** auf **Gemessen** zurück und zeigt den Hinweis „Wahrheit ausgeblendet“. Was bleibt: `measurements`, `observed_summary`, die Betreiber-Konfiguration (`producers/storages/controls`) — und die `estimated`-Ebene, denn sie ist aus Messwerten rekonstruiert und gehört legitim dem Betreiber (Abschnitt 5.3). Damit lassen sich Übungen abnahmesicher stellen („Schätzen Sie den Rücklauf am Netzeende“), ohne dass die Lösung per API abgreifbar wäre.

5.3. Der Vorwärts-Beobachter (die Schätzung)

`pandapipes` hat *keinen* Zustandsschätzer — nichts wie die gewichtete Kleinst-Quadrat-Schätzung (WLS) von `pandapower`. Die Sicht **Schätzung** ist deshalb keine WLS-Rekonstruktion, sondern ein **Vorwärts-Beobachter** (ein „digitaler Zwilling“): ein zweites, tief kopiertes `pandapipes`-Netz, das mit derselben Physik (und derselben Retry-Ladder) gelöst wird, aber *nur mit Betreiberwissen* gefüttert:

- die SCADA-Vorlauftemperatur der Zentrale und der vom Betreiber gesetzte Pumpen-Sollwert (`plift`);
- die Messwerte der platzierten Zähler, *modus-treu* (ein Standard-Zähler im Kaltstart liefert `null` → der Prior übernimmt diesen Kanal);
- für alle **unbemessenen** Abnehmer ein **Erwartungsprofil** (Prior): das deterministische Archetyp-Profil, wettergestützt am Tag gewählt (eine Gradstunden-Lastprognose: erwartete Tagesenergie → nächster Archetyp-Tag), mit dem Mittel des Warmwassers über die stochastischen Varianten;
- die Anlagen-Sollwerte des Betreibers (Erzeuger, Speicher — das ist Konfiguration, immer sichtbar);
- die wahre Außentemperatur inklusive Override (die Zentrale hat eine Wetterstation, und der Override ist der eigene Regler des Betreibers).

Entscheidend: Die Prioren lesen den *unveränderlichen Planungsvertrag* (`sim.inputs`), den Archetyp-Cache und die Platzierungs-Rezepte — **niemals** die veränderlichen Laufzeit-Arrays (`sim.profiles`), die die stochastische Minutenwahrheit tragen. Eine zur Laufzeit injizierte Anomalie kann deshalb prinzipiell nicht in den Prior lecken. Genau diese Disziplin prüfen die Stolperdrähte (Abschnitt 5.4). Der Prior-Modus ist über `POST`

/estimation/config wählbar: **archetype** (Vorgabe, das treuere Erwartungsprofil) oder das bewusst grobe **design** (Auslegungslast $\times$ Gradstundenfaktor, ohne Warmwasser).

Schätzgüte: die Innovation

Die Übersicht zeigt in der Schätzsicht ein **Güte-Abzeichen**. Der Fehler **estimated.error** ist die *Innovation* des Beobachters: $|\text{Zwilling} - \text{Messung}|$ an jeder *bemessenen* Stelle — Rücklauftemperatur, Massenstrom und Δp an Zählern; Rücklauf und Paar- Δp an T/p-Sensoren; Rücklauf und Massenstrom an der SCADA. Er wird also gegen *Messwerte* berechnet, nicht gegen die Wahrheit — deshalb ist er auch im strikten Modus gültig und sichtbar. Aggregiert erscheinen max / mean von $|\Delta T_{\text{Rücklauf}}|$, $|\Delta \dot{m}|$ und $|\Delta \Delta p|$, dazu das Alter der Schätzung (in Simulationsminuten, mit Sequenznummer **#seq**) und die Rechenzeit des Beobachters.

Volle Abdeckung kauft Gewissheit Mit der Vorlage **Alle Abnehmer** im Modus **Voll** ist der Zwilling an jedem Abnehmer durch die gemessenen Randbedingungen (Leistung + Rücklauf) und an der Zentrale durch die gemessene Vorlauftemperatur festgenagelt — die gesamte gekoppelte Lösung ist bestimmt. Die Schätzung trifft die Wahrheit dann praktisch exakt (Einspeisung auf 0,1 kW, Massenstrom auf 10^{-3} kg/s), die Innovation geht gegen null. Entfernt man Zähler, treten Priors an die Stelle von Sensoren, und die Innovation wächst — diese monotone Beziehung ist das quantitative Gesicht von „Abdeckung kauft Gewissheit“.

Was eine Schätzung kostet (Drosselung)

Der Beobachter ist ein zweites volles Netz — wenn er rechnet, verdoppelt er grob die Physikkosten des Schritts. Er rechnet deshalb *nicht* jeden Tick. Zwei Tore werden UND-verknüpft: das **Messraster** (im Standard-Modus mit platzierten Geräten nur an den 15-Minuten-Fenstergrenzen — dazwischen gibt es keine neue Information) und eine **Wanduhr-Selbstdrosselung** (eine neue Schätzung erst, nachdem **throttle_factor** $\times$ ihre eigene letzte Rechenzeit verstrichen ist, Vorgabe 2,0). Zwischen den Aktualisierungen reitet die letzte Schätzung auf jedem Frame mit, ehrlich mit **step/day/seq** gestempelt, sodass die Oberfläche ihr Alter zeigen kann.

Sichtbare Rechenzeit auf dem vermaschten Netz Auf einem kleinen Lehrnetz kostet der Zwilling nur wenige zehn Millisekunden. Auf dem vermaschten Verbier-Netz (1352 Knoten, Tier-2-Solver, ≈ 5 s je Lösung) hält die Selbstdrosselung ihn davon ab, überhaupt jeden Tick zu laufen: Die Schätzsicht zieht in größeren Sprüngen nach. Das ist kein Fehler, sondern sichtbare Rechenzeit — Beobachtbarkeit hat neben den Gerätekosten auch Rechenkosten, und beide wachsen mit der Netzgröße.

5.4. Ehrlichkeits-Stolperdrähte

Die Stolperdrähte sind die Tests, die den Beobachtbarkeitsvertrag *falsifizierbar* machen. Jeder injiziert eine Lage, in der ein naiver Schätzer schummeln würde, und prüft, dass er es nicht tut.

Eine unbemessene Anomalie bleibt unsichtbar Man bemisst nur Abnehmer B und injiziert an Abnehmer A (unbemessen) eine Störung direkt in die Laufzeitwahrheit: verdoppelte Nachfrage und +15 K Rücklauf. Die **Realität** zeigt A prompt mit über 155 kW und über 68 °C Rücklauf — die Anomalie *ist* in der Wahrheit. Die **Schätzung** bleibt beim Prior (rund 80 kW Plan, unter 57 °C), denn kein Sensor deckt A ab und die Prioren lesen den unveränderlichen Vertrag, nicht die Laufzeit-Arrays. Verschiebt man denselben Zähler auf A, *propagiert* dieselbe Anomalie sofort (die Schätzung zeigt über 155 kW). Zusammen beweisen beide Fälle: Die Schätzung trägt exakt die Information, die die Sensorik liefert — nicht mehr und nicht weniger. *So viele Wärmemengenzähler braucht die Schätzung, um eine Rücklauf-Anomalie zu sehen: mindestens einen — an genau dieser Station.*

Ohne Zähler ist die Schätzung der Prior Unter der Vorlage **Alles entfernen** (nur SCADA) ist jeder geschätzte Abnehmerwert exakt gleich seinem Prior — der Zwilling wird von nichts anderem getrieben. Die Schätzungsgüte (z. B. die Innovation des Erzeuger-Massenstroms) zeigt dann ehrlich, wie weit die Erwartung von der Wahrheit abweicht: Am Demo-Dorf um 08:00 Uhr steigt die Massenstrom-Innovation von 0,000 (**Alle Abnehmer**) über halbe Abdeckung bis 0,189 kg/s (**Alles entfernen**) — die Degradation ist in genau der Zahl sichtbar, die die Oberfläche anzeigt.

5.5. Der Blindflug der Schlechtpunktregelung

Hier treffen sich Beobachtbarkeit und Regelung. Die Schlechtpunktregelung (Kapitel 6) liest **ausschließlich** `observed_summary.dp_worst_bar` — das Minimum des Δp über die *bemessenen* Abnehmer — niemals die Wahrheit. Daraus folgt eine Degradationsleiter:

1. **Kein verwertbarer Δp -Messwert** (Vorlagen **Nur Erzeuger/Alles entfernen**, oder alle Zähler noch im Standard-Kaltstart) → der Wert ist `null` → die Regelung **hält** die Förderhöhe. Halten *ist* „auf den Zentralen- Δp regeln“: eine Konstant- Δp -Pumpe ohne Fernfühler.
2. **Bemessen, aber der wahre Schlechtpunkt trägt keinen Zähler** → die Regelung regelt den besten *gemessenen* Punkt (was ein realer Schlechtpunktreger auch tut) und der Frame setzt `controls.dp_control.blind_spot = true`.

Das `blind_spot`-Flag ist bewusste Meta-Information über die *Eignung der Sensorplatzierung*: Es benennt keinen Physikwert, nur einen Booleschen Zustand darüber, ob die Sensorik den wahren Schlechtpunkt abdeckt — und bleibt deshalb auch im strikten Modus sichtbar (es teacht den Betreiber, einen Zähler zu *versetzen*, nicht der Zahl zu misstrauen). Der Abschnitt **Slechtpunkt** zeigt zwei verschiedene Hinweistexte für die beiden Fälle.

Der wandernde Schlechtpunkt Mit der Vorlage **Schlüsselstellen** liegt der Zähler dort, wo der Schlechtpunkt beim letzten Rechenschritt *bekannt* war — nicht notwendig dort, wo er *jetzt* ist. Am Demo-Dorf pendelt der wahre Schlechtpunkt unter der Warmwasser-Streuung zwischen zwei Abnehmern ($C9 \leftrightarrow C5$). Das `blind_spot`-Flag geht genau dann auf `true`, wenn der unbemessene C9 kritisch wird — die Gemessen-Ebene meldet dann ehrlich den bemessenen C5 bei einem *höheren* Δp . Man sieht die Flagge kommen und gehen: die Lektion vom wandernden Schlechtpunkt, live auf dem Bildschirm. Das ganze Drei-Sichten-Modell existiert, um genau das sichtbar zu machen.

6. Physik und Regelungen

Dieses Kapitel erklärt, was der Solver rechnet und was jeder Regler-Knopf tut. Alle Formeln und Zahlen sind das, was der Code an **master** und seine bekannte Antwort-Referenz tatsächlich produzieren.

6.1. Das Vor-/Rücklaufmodell

pandapipes ist ein stationärer Rohrströmungslöser. rheatflow modelliert ein Zweileiternetz — heißer Vorlauf, kühler Rücklauf —, indem der Netzbauer *jeden* Trassenknoten und *jede* Trasse in ein **Paar** von pandapipes-Elementen entfaltet, und zwar *einmal* pro Konfiguration:

- ein Trassenknoten → zwei **junctions** (<name>_s Vorlauf, <name>_r Rücklauf), beide initialisiert auf die Vorlauftemperatur der Zentrale;
- eine Trasse → zwei **pipes** (Vorlauf from→to, Rücklauf to→from, zur Zentrale hin umgekehrt);
- ein Abnehmer → ein **heat_consumer** (Vorlauf→Rücklauf-Knoten), zieht **qext_w**;
- die Zentrale → genau eine **circ_pump_const_pressure** (der Druck-Slack, Rücklauf→Vorlauf).

Jeder Tick überschreibt nur die Elementwerte aus dichten [Elemente, Ticks]-Arrays und löst neu — das Netz wird nie neu gebaut.

Zwei Bau-Invarianten. Erstens: Temperaturen sind *Kelvin im Solver, Grad Celsius auf dem Draht* (Rundung auf 6 Stellen, NaN/±Inf → **null**). Zweitens: **text_k** (die Erdreich-Temperatur am Rohr) wird auf *jedem* Rohr explizit gesetzt — der Signatur-Standard 0 hieße 0 K Umgebung und würde die Verluste etwa vervierfachen. Jeder Tick frischt **text_k** aus der Erdreich-Zeitreihe auf.

6.2. Die bidirektionale Lösung und die Retry-Ladder

Temperaturgeregelter Abnehmer machen den Massenstrom von der *ankommenden* Temperatur abhängig (ein Abnehmer mit Rücklaufvorgabe muss bei kühlerem Vorlauf mehr Massenstrom ziehen, um dieselbe Leistung zu liefern). Das koppelt Hydraulik und Wärme,

deshalb ist `mode="bidirectional"` die Vorgabe. Konvergiert ein Schritt nicht sofort, geht der Löser eine vierstufige **Retry-Ladder** von oben nach unten durch:

Stufe	Aufruf	Zweck	Status
1	<code>bidirectional, iter=N</code>	Normallösung	ok
2	<code>bidirectional, iter=N, alpha=0.5</code>	Halbschritt-Dämpfung	ok
3	<code>bidirectional, iter=2N, alpha=0.2</code>	starke Dämpfung	ok
4	<code>sequential, iter=N</code>	entkoppelt (Sollwerte nicht gehalten)	degraded

N ist `RTHEATFLOW_SOLVER_ITER` (Vorgabe 100). `nonlinear_method="automatic"` wird bewusst *nie* verwendet (es löst unter `bidirectional` in 0.14.0 einen Broadcast-Fehler aus). **Nichtkonvergenz ist Daten, kein Absturz:** Scheitern alle Stufen, wird der letzte konvergierte Zustand weiterverwendet und ein Frame mit `converged=false` / `solver_status="failed"` veröffentlicht — die `asyncio`-Schleife lebt weiter, ein Solverproblem erscheint als Feld und UI-Abzeichen, nie als HTTP 500. Nach jedem konvergierten Schritt wird der gelöste Knotenzustand zur nächsten Initialisierung (*Warmstart*) — das hält warme Lösungen bei $\approx 20\text{--}35$ ms.

6.3. Abgeleitete Größen: Verluste und Einspeisung

`pandapipes` liefert rohe Ergebnistabellen; die Plattform rechnet jede berichtete Kennzahl selbst (in `collect_physics`, geteilt von Wahrheit *und* Zwilling — ein Formelsatz, nie zwei).

Warum die Verlustformel richtungsabhängig ist Der Verlust eines Rohres ist $q_{\text{loss}} = |\dot{m}| \cdot \bar{c}_p \cdot (T_{\text{ein}} - T_{\text{aus}})$. Der Kniff: *welches Ende ist der Einlass?* Auf einem vermaschten Netz (Verbiers sechs Schleifen) kann ein Rohr in beide Richtungen fließen, und die Rücklaufrohre fließen ohnehin „rückwärts“ zu ihrer `from→to`-Definition. Die Formel liest den Einlass aus dem Vorzeichen des Massenstroms ($T_{\text{ein}} = t_{\text{from}}$ falls $\dot{m} \geq 0$, sonst t_{to}). Eine naive Formel, die immer `t_from` als Einlass nimmt, bekommt auf jedem rückwärts fließenden Ast das Vorzeichen falsch und **überzählt den Gesamtverlust um $\approx 20\%$** auf vermaschten Netzen.

Einspeisung: $\dot{m} \cdot \bar{c}_p \cdot \Delta T$, **nicht die Enthalpiespalte** Die Einspeisung der Zentrale wird aus dem Ergebnis der Slack-Pumpe als $\dot{m} \cdot \bar{c}_p \cdot \Delta T$ berechnet — *nicht* aus der Spalte `res_circ_pump_pressure.qext_w`. Jene ist eine Enthalpieform, die etwa $+4,7\%$ zu hoch liest und die Energiebilanz nicht schließt: Auf der bekannten Antwort-Referenz sind es **187,182 kW** ($\dot{m}\bar{c}_p\Delta T$) gegen 195,965 kW (Rohspalte). Die Plattform berichtet den bilanzschließenden Wert; `summary.balance_err_kw` ist der Ehrlichkeits-Stolperdraht pro Schritt (auf der Referenz $-0,04\%$).

6.4. Die Heizkurve

Die Heizkurve ist die Vorlauf-Temperaturregelung der Zentrale: kälter draußen → heißerer Vorlauf (gleitender Betrieb). Sie folgt einer gleitenden Kurve mit Formexponent n :

$$t_{\text{flow}}(t_{\text{amb}}) = \text{clamp}\left(t_{\text{flow,min}} + (t_{\text{flow,design}} - t_{\text{flow,min}}) \cdot r^{1/n}, t_{\text{flow,min}}, t_{\text{flow,design}}\right),$$

$$r = \max\left(0, \frac{t_{\text{room}} - t_{\text{amb}}}{t_{\text{room}} - t_{\text{amb,design}}}\right)$$

mit den Vorgaben $t_{\text{amb,design}} = -12^\circ\text{C}$, $t_{\text{flow,design}} = 110^\circ\text{C}$, $t_{\text{flow,min}} = 70^\circ\text{C}$, $t_{\text{room}} = 20^\circ\text{C}$, $n = 1$ (linear; $\approx 1,3$ entspricht der Heizkörper-Charakteristik). Zwei **Presets** machen die Temperaturabsenkungs-Geschichte mit einem Klick sichtbar:

- **3G** — `t_flow_design=110`, `t_flow_min=70` (ein klassisches 110/60-System);
- **4G** — `t_flow_design=70`, `t_flow_min=65` (ein Niedertemperatur-70/40-System).

Ein Netz ohne Heizkurven-Block läuft bei konstanter Vorlauftemperatur (die Heizkurve ist dann null im Frame — mehrere Referenznetze wie DESTEST tun das). Dieselben Auslegungspunkte $t_{\text{room}}/t_{\text{amb,design}}$ verankern auch den Gradstunden-Faktor der Nachfrageskalierung (Kapitel 8) — die Physik bleibt konsistent, wenn der Betreiber am Wetterregler zieht.

Wie sich die Verlustquote zwischen 3G und 4G ändert Auf dem Appendix-A-Netz bei $t_{\text{amb}} = 0^\circ\text{C}$ macht ein Umschalten von 3G auf 4G beide gegenläufigen Effekte quantitativ:

Preset	t_{flow}	Verluste	Massenstrom Zentrale	Pumpe P_{el}
3G (110/70)	95,0 °C	31,10 kW (16,3 %)	1,152 kg/s	0,34 kW
4G (70/65)	68,1 °C	21,35 kW (11,8 %)	2,199 kg/s	0,64 kW

Die Verluste sinken um 31 %, weil ein kühleres Vorlaufrohr weniger an das Erdreich verliert. Aber: Für eine feste Nachfrage $Q = \dot{m} c_p \Delta T$ schrumpft bei kühlerem Vorlauf (und nicht mitsinkendem Rücklauf) das ΔT , also muss der Massenstrom steigen — hier $\times 1,91$ — und der Pumpenstrom (mit $P_{\text{hyd}} = \dot{V} \cdot \Delta p$, quadratisch im Fluss) nahezu verdoppelt sich. Das ist das *Low- ΔT -Syndrom* (Niedertemperatur-Syndrom), direkt sichtbar in der Geschwindigkeits-Ebene der Karte und in den Übersichts-Kennzahlen.

6.5. Die Schlechtpunktregelung

Die Förderhöhe der Zentralen-Pumpe (`plift_bar`) wird geregelt, um den *kritischen* Abnehmer versorgt zu halten. Der Regler macht **einen begrenzten Proportionalschritt pro Tick, nach der Lösung** — bewusst keine innere Iterationsschleife:

$$\text{plift} += \text{clamp}(K \cdot (\Delta p_{\text{Soll}} - \Delta p_{\text{worst}}), \pm \text{max_step})$$

mit $K = 0,5$, $\text{max_step} = 0,05$ bar/Tick, Sollwert-Band 0,3–2,0 bar (Vorgabe 0,7 bar) und absoluter Förderhöhen-Grenze 0,1–10 bar. Die aktualisierte Förderhöhe wirkt erst auf die Lösung des *nächsten* Ticks — die Pumpe reagiert also *über Minuten* wie eine echte drehzahlgeregelte Pumpe, und die Engine bleibt bei einer Lösung pro Tick ($O(1)$, keine Fixpunkt-Schleife). Der Modus **fest** („ungeregelte Pumpe“) hält die Förderhöhe, wo der Nutzer sie gesetzt hat — der Lehr-Kontrast, der die Überdimensionierung einer Konstant-Förderhöhen-Pumpe zeigt. Dass der Regler *nur* die Gemessen-Ebene liest, ist der Kern der Blindflug-Semantik (Abschnitt 5.5).

Was passiert mit dem Schlechtpunkt- Δp , wenn morgens alle Stationen aufheizen?

Fährt die morgendliche Nachfrage hoch, steigen die Massenströme und damit die Druckverluste im Netz; der Δp am ungünstigsten Abnehmer fällt. Der Regler sieht das (über den bemessenen Schlechtpunkt) und hebt die Förderhöhe — aber nur um 0,05 bar je Tick. In der M4-Abnahme trieb er den beobachteten Schlechtpunkt von 1,953 bar streng monoton auf den Sollwert 0,7 bar, Setzung nach ≈ 26 Ticks bei 0,703 bar, dabei fiel der Pumpenstrom von 0,39 auf 0,14 kW. Mit der Vorlage **Schlüsselstellen** lässt sich zusehen, wie der Regler die Morgenrampe über Minuten nachfährt — und wie das `blind_spot`-Flag flackert, wenn der wahre Schlechtpunkt wandert (Abschnitt 5.5). Ein überhöhter Sollwert kostet messbar Pumpenstrom; der Modus **fest** zeigt den Unterschied.

7. Anlagen

Alle Anlagen lassen sich zur Laufzeit über das Element-Kontextmenü (Rechtsklick → „Hier platzieren“) setzen und über den angehefteten Abschnitt (Strg-Klick) konfigurieren. Sie wirken sofort im laufenden Netz; ein einzelner nicht konvergierter Schritt heilt sich im nächsten Takt selbst.

7.1. Erzeuger

pandapipes weiß nichts von Kesseln oder Wärmepumpen — der Slack ist nur eine Druck-/Temperaturquelle. Die *Erzeugerart* ist ein Dispatch-Modell *über* dem Slack, das aus der gelösten Einspeisung und den Live-Temperaturen pro Tick die Strom-/Brennstoffseite berechnet:

Art	Formel	Anmerkung
Kessel (boiler)	$p_{\text{fuel}} = q/\eta$	η Vorgabe 0,95
BHKW (chp)	$p_{\text{el}} = \text{pq} \cdot q$	wärmegeführt, pq Vorgabe 0,5
Wärmepumpe (heat_pump)	$\text{COP} = \eta_g \cdot T_{\text{hot}}/(T_{\text{hot}} - T_{\text{cold}})$	$p_{\text{el}} = q/\text{COP}$

T_{hot} ist die *live* Vorlauftemperatur der Zentrale, T_{cold} die live Quelltemperatur (t_{amb} für eine Luft-, t_{ground} für eine Erdreich-Wärmepumpe); der Gütegrad η_g (Vorgabe 0,5) skaliert den idealen Carnot-COP, der Temperaturhub ist bei 1 K abgefangen. Die Kennzahlen werden jeden Tick neu gerechnet und erscheinen im `producers[]`-Block neben `q_kw`, `t_flow_c`, `plift_bar` und `pump_el_kw`.

Die Wärmepumpe belohnt niedrige Netztemperaturen T_{hot} ist die live Vorlauftemperatur — senkt die Heizkurve den Vorlauf, steigt der COP und die berichtete elektrische Leistung sinkt. Das ist die Temperaturabsenkungs-Geschichte, quantitativ: Das Absenken der Netztemperaturen hebt den COP um grob 2–3 % je Kelvin. Ein 4G-Netz mit einer Erdreich-Wärmepumpe fährt deshalb sichtbar sparsamer als dasselbe Netz auf 3G — genau das vergleicht das mitgelieferte Szenario „Demo-Dorf 4G Vergleich“ (Kapitel 10), das eine Erdreich-Wärmepumpe fährt.

7.2. Pufferspeicher

Ein Pufferspeicher (Vorgabe 100 kWh / 50 kW) überbrückt das Vor-/Rücklaufpaar eines Knotens mit *zwei* Zweigen, von denen pro Tick genau einer aktiv ist:

Laden ein `heat_consumer` mit `qext_w + treturn_k` = Speicher-Untertemperatur (Vorgabe 45 °C): zieht Ladeleistung aus dem Netz.

Entladen eine `circ_pump_const_mass_flow` (druckfrei, `type="t"`) drückt einen festen Massenstrom bei der Speicher-Obertemperatur (Vorgabe 80 °C) in den Vorlauf. Die druckfreie Variante ist zwingend — eine druckfixierende Pumpe würde gegen das Druckfeld des Slacks kämpfen und alle Retry-Stufen zum Divergieren bringen.

Bereitschaft der Ladezweig fällt auf das Nullfluss-Bodenpaar (0,02 kg/s + 100 W), die Entladepumpe geht außer Betrieb — die Stummelrohre erreichen nie Nullfluss.

Der Ladestand (SoC) wird aus den *gelösten* Ergebnissen integriert (Laden über die realisierte `qext_w`, Entladen über $\dot{m} c_p \Delta T$), mit Einweg-Wirkungsgrad $\eta = 0,95$ (Umlauf $\eta^2 \approx 0,90$); die Bereitschaft ist ein stehender Verlust, der SoC bewegt sich dann nicht.

Warum ein voller Speicher von selbst aufhört zu laden `desired_state` wendet Leistungs- und Kapazitätsgrenzen an, bevor der aktive Zweig feststeht: Eine Ladeanforderung wird durch den verbleibenden Raum gedeckelt, eine Entladeanforderung durch die verfügbare Ladung; unter 1 W idlet der Speicher von selbst. Ein voller Speicher stellt das Laden also selbsttätig ein, ein fast leerer klingt beim Entladen geometrisch ab und geht dann in Bereitschaft. Zum Kappen der morgendlichen Wärmespitze lädt man den Speicher nachts (niedrige Last, günstige Erzeugung) und entlädt ihn zur Morgenrampe — der SoC-Balken im angehefteten Abschnitt zeigt das live.

7.3. Einspeiser und Bypass

Einspeiser (Solarthermie/Abwärme, Vorgabe 20 kW) sind Wärmetauscher: eine feste Einspeisung als negatives `qext_w`. Setzt man einen 20-kW-Einspeiser, fährt die Zentrale entsprechend zurück (verifiziert: 187,2 → 173,8 kW, Bilanz 0,07 %). Die Plattform bleibt ein Zweileiternetz mit *einem* Druckhalter — weitere Erzeuger speisen als Wärmetauscher oder Massenströmpumpen ein, nie als zweiter Druck-Slack (der zweite Slack quittiert mit HTTP 409).

Bypass (Netzschluss) ist das kanonische Massenzstrom-Paar: ein winziger fester Massenzstrom (0,02 kg/s) plus eine kleine Bereitschaftswärme (100 W), gesetzt, um ein Endstrang durchströmt und warm zu halten.

Der Bypass hält den Endstrang warm Ein Trassenknoten mit Grad 1 (ein Endstrang ohne Abnehmer) trägt ohne Anschluss Nullfluss — und ein Nullfluss-Ast ist in der

Wärme-Matrix von pandapipes singulär. Der Bypass löst das: sein fester Massenstrom trägt keine Singularität. Seine Bereitschaftswärme parkt im Warmwasser-Slot, damit der Wetter-Override sie nie umskaliert — die Aufgabe eines Bypasses ist Temperaturhaltung, nicht wetterfolgende Nachfrage. Das reale Ortsnetz Schutterwald setzt genau diesen kanonischen Bypass an seine abnehmerlosen Zweige.

7.4. Die Nullfluss-Schwelle

Ein `heat_consumer` mit null `qext_w` trägt null Massenstrom, und die Nahe-Null-Last ist genau das Regime, in dem die `qext_w+treturn_k`-Regelung divergiert (es gibt keinen Fluss, dessen Rücklauftemperatur man regeln könnte). Die Schwelle `RTHEATFLOW_MIN_QEXT_W` (Vorgabe 500 W) hebt die Gesamtnachfrage jedes *temperaturgeregelten* Abnehmers auf diesen Boden. Entscheidend: Die Schwelle ist *zeilenweise und maskengerecht* — Massenstrom-Paar-Zeilen (Bypässe, festfließende Abnehmer) sind ausgenommen, ein 100-W-Bypass bleibt 100 W. Der Vorgabewert 500 W ist ein bewusst großzügiger Löser-Schutz für den interaktiven Betrieb; die Validierungsläufe senken ihn unter das reale Minimum des jeweiligen Netzes (DESTEST 62 W).

8. Der Wetter-Regler

Eine Wetter-Zeitreihe treibt alles: Die Außentemperatur speist die Raumheizlast und die Heizkurve, die Erdreich-Temperatur speist `text_k` jedes Rohres. Der Abschnitt **Wetter** in der Seitenleiste zieht die Außentemperatur live (-30 bis $+45$ °C, per PUT `/weather/override`, 150-ms-entprellt); Loslassen kehrt zum Wetterprofil zurück.

8.1. Die Gradstunden-Skalierung

Der Mechanismus ist der Gradstunden-Faktor $f(T) = \max(0, T_{\text{room}} - T) / (T_{\text{room}} - T_{\text{design}})$, angewandt *nur* auf die Raumheizung — das Warmwasser (DHW) bleibt **unberührt**, weil die Warmwassernachfrage dem Wetter nicht folgt. Zwei Regime, abgesichert durch $\varepsilon = 0,05$:

- **Normal:** Das Profil hat nennenswerte Raumheizung ($f_{\text{Profil}} \geq \varepsilon$) $\rightarrow$ skaliere es mit dem Verhältnis $f(T_{\text{override}}) / f(T_{\text{Profil}})$ (die Tagesform bleibt erhalten).
- **Sommer:** Die Raumheizung des Profils ist ≈ 0 ($f_{\text{Profil}} < \varepsilon$) $\rightarrow$ es gibt kein Verhältnis zum Skalieren, also baue die Nachfrage aus der Auslegungslast neu auf, $q_{\text{design}} \cdot f(T_{\text{override}})$.

Der Override setzt nur die effektive Außentemperatur; die Erdreich-Temperatur wird nie überschrieben (eine langsame, saisonale Größe). Weil dieselben Auslegungspunkte auch die Heizkurve verankern, reagieren Vorlauftemperatur *und* Nachfrage konsistent auf den Regler.

Einen Sommertag in den Winter ziehen Startet man an einem Sommertag (Raumheizung nahe null) und zieht den Wetterregler auf -10 °C, greift das Sommer-Regime: Es gibt keine Tagesform zum Skalieren, also rekonstruiert die Plattform die Heizlast aus der Auslegungslast mal Gradstundenfaktor — man bekommt eine physikalisch plausible Winterlast aus einem Nahe-Null-Profil. So lassen sich Kälteeinbruch und Sommerbetrieb im Zeitraffer durchspielen, ohne das Netz zu wechseln.

Warum die *relative Verlustquote* im Sommer hochspringt Die Wärmeverluste eines Netzes sind fast nachfrageunabhängig (die stehenden Verluste des Rohrs bleiben, ob viel oder wenig Wärme fließt). Zieht man den Wetterregler in den Sommer, bricht die Nachfrage ein — die *absoluten* Verluste sinken kaum, die *relative* Verlustquote (Verlust/Einspeisung) springt sichtbar hoch. Am realen Schutterwald reicht dieselbe Physik von $\approx 7,5$ % im

tieften Winter bis $\approx 45\%$ im Sommer. Das ist derselbe Effekt wie die Abhängigkeit von der Wärmedichte — nur in der Zeit statt im Raum.

9. Referenznetze und validierte Physik

Neben den Lehrnetzen enthält der Katalog vier Netze aus realen bzw. publizierten Quellen. Jedes Netzverzeichnis unter `data/networks/<id>/` trägt eine `DATASET.md` mit Herkunft, Lizenz, Konvertierungsentscheidungen und dem vollständigen Validierungsprotokoll — und alle hier genannten Zahlen sind *live nachgerechnete* Werte, gegen *externe* Wahrheit geprüft, mit vorab (nicht nachträglich) begründeten Toleranzbändern.

9.1. Der Katalog

id	Charakter	Knoten	Trasse km	Rolle
demo_dorf	rural	15	0,953	Karten-Demo (Standardnetz)
appendix_a	abstract	4	1,0	bekannte Antwort-Referenz
schutterwald	rural	208	2,626	reales Ortsnetz (Vorzeigenetz)
destest_16	benchmark	25	0,39	validiert (Tabelle 6)
destest_8	benchmark	13	0,216	Katalogbeispiel
destest_32	benchmark	49	0,792	Katalogbeispiel
verbier	meshed	676	12,05	vermascht, Messdaten

Die Lehrnetze. `demo_dorf` ist ein handgefertigtes Straßendorf auf realer Geographie (nördlich von Karlsruhe): eine „Heizzentrale“, 11 Abnehmer (6×EFH_ALT, 3×EFH_SAN, 2×MFH), 182 kW Auslegung, 3G-Heizkurve — es existiert, damit die Karte etwas Reales zeichnen kann (lineare Wärmedichte bewusst unter der Faustregel, siehe Abschnitt 3.1). `appendix_a` ist die abstrakte Vier-Knoten-Referenzschleife, deren Abnehmer A/B/C *alle drei* Partnertypen ausüben (Rücklauf/ ΔT /Massenstrom); ihr Slack trägt *keine* Heizkurve, damit die Antwort deterministisch ist — die Testsuite pinnt an ihr die bekannten Zahlen (A *in* 0,674357, C *in* 0,25, Verluste 27,258 kW, Einspeisung 187,182 kW, Bilanz −0,04 %).

9.2. DESTEST — der validierte Cross-Tool-Benchmark

Die IBPSA-„DESTEST“-Übungen sind ein Cross-Tool-Benchmark: mehrere unabhängige Werkzeuge modellieren dasselbe kleine Fernwärmenetz und veröffentlichen ihre Ergebnisse, sodass die Streuung zwischen den Werkzeugen die Toleranz ist. `destest_16` (16 Gebäude, Netz CE_1, aus der Fallbeschreibung Tabelle 6 gebaut) wird gegen zwei Übungen

validiert. Der Solver läuft mit $\text{min_qext_w} = 62 \text{ W}$ — bei ΔT 30 K ist das die 1,77-kg/h-Mindestumgehung von DESTEST, der Nullfluss-Boden *ist* die Bypass-Regel des Falls.

CE 0 — eingeschwungener Zustand (alle 16 Stationen exakt auf Spitzenlast, gegen 6 Werkzeuge):

Kennzahl	rheatflow	publiziert (6 Tools)	Urteil
Massenstrom Zentrale	8876,5 kg/h	8847,9–8870,4	+0,07 % über Max
Rücklauftemperatur	39,48 °C	39,46–39,85	im Band
Wärme gesamt	314,8 kW	308,2–314,3	+0,16 % über Max
Vorlauf an SD1	69,454 °C	69,43–69,48	im Band

Die beiden Kennzahlen knapp über dem publizierten Maximum (Massenstrom und Wärme) sind genau die, die der Fluid-Eigenschaftsunterschied bewegen soll: pandapipes' temperaturabhängiges Wasser läuft mit $c_p \approx 4179\text{--}4187 \text{ J}/(\text{kg}\cdot\text{K})$ gegen DESTESTs feste 4180, und ein ΔT -geregelter Massenstrom skaliert invers mit c_p . Die Vorlauftemperatur an SD1 — reine Verlustphysik, vom Fluid-Konstanten unabhängig — landet *im* Band: der eigentliche Prüfstein, dass das mehrschichtige Wandverlustmodell die publizierten Rohrverluste reproduziert.

CE 1 — die Sieben-Tage-Woche (1008 Lösungen auf dem 10-min-Raster, gegen 3 dynamische Werkzeuge):

Kennzahl	rheatflow	AixLib	Buildings	IBPSA
Wärme-Einspeisung	14,506 MWh	14,452	14,324	14,357
Netzverluste	606,6 kWh	535,2	543,8	535,7

Die ehrliche Nacht-Zuordnung der Verluste Der Verlust-Gesamtwert liegt $\approx 12 \%$ über dem publizierten Maximum — und das ist die lehrreichste ehrliche Zuordnung der Plattform. Die Woche zerfällt sauber: Die *belasteten* Ticks (Nachfrage $>5 \text{ kW}$) tragen **534 kWh** Verlust — *innerhalb* des publizierten Bandes 535–544 kWh. Unter Last stimmt die Verlustphysik mit den dynamischen Rohrwerkzeugen überein. Der **Überschuss von +72,5 kWh** sitzt komplett in den **401 Nahe-Null-Nachtticks**: Dort kühlen die publizierten dynamischen Rohre *ab*, während ein quasistatischer eingeschwungener Zustand die Rohreintritts-Region auf Vorlauftemperatur hält und deshalb den stehenden Verlust überzählt. Der Überschuss ist nicht frei — er ist *vor* der Messung durch die Enthalpie der DESTEST-Mindestumgehung nach oben begrenzt ($16 \times 1,77 \text{ kg/h} \times c_p \times 60 \text{ K} \approx 2 \text{ kW}$ über 401 Ticks $\Rightarrow \leq 134 \text{ kWh}$). Das Band ist also asymmetrisch und *hergeleitet*, nicht getunt.

9.3. Schutterwald — Plausibilität am realen Ort

Schutterwald ist ein realer Ort (Baden) ohne publizierte Cross-Tool-Antwort, deshalb wird es auf *Plausibilität* geprüft: geschlossene Energiebilanz, Geschwindigkeiten im Band, Jahres-Verlustquote im Literaturbereich. 208 Trassenknoten, 207 Trassen, 44 Übergabestationen aus dem realen Gasnetz abgeleitet (50-m-Anschlussradius), 2,626 km reale WGS84-Trasse, 1904 kW Auslegung, LHD 1,34 MWh/(m·a), 3G-Kurve. Das Karten-Vorzeigenetz.

Metrik	Wert	Prüfung
Konvergenz	alle 96 Schritte Tier 1	<code>solver_status=ök</code>
worst-case Bilanzfehler	0,075 % der Einspeisung	≤ 1 %
Spitzengeschwindigkeit	0,98 m/s	$< 1,3$ m/s
Wintertag-Verlustquote	7,5 % der Einspeisung	$0,04 < r < 0,12$

Die Verluste sind nahezu nachfrageunabhängig (≈ 61 kW etwa konstant), also annualisiert die instantane Winterquote von 7,5 % höher: $61 \text{ kW} \times 8760 \text{ h} \approx 537 \text{ MWh}$ Verlust gegen $\approx 3,52 \text{ GWh}$ Nachfrage $\Rightarrow \approx \mathbf{13}$ % Jahresverlustquote — im 8-20-%-Plausibilitätsband für ein 3G-Netz dieser Dichte. Die Unterscheidung instantan/annualisiert ist der Punkt: Ein einzelner kalter Tag unterschätzt die Jahresverluste.

9.4. Verbier — gegen reale Messdaten

Verbier (OpenDHN, Zenodo DOI 10.5281/zenodo.10793816, CC BY 4.0) ist das einzige Referenznetz, das gegen *reale eingeschwungene Messdaten* validiert wird — ein reales vermaschtes Alpennetz: 676 Trassenknoten, 681 Trassen je Seite (12,05 km, **6 unabhängige Schleifen**), zwei Heizzentralen, 150 Übergabestationen mit gemessener Leistung und gemessenem Rücklauf. Die gemessenen *Vorlauf*temperaturen sind *nie* Eingabe — sie sind das Validierungsziel. Der vermaschte 1352-Knoten-Netz mit druckfreier zweiter Pumpe scheitert an Tier 1 und konvergiert erst auf **Tier 2** ($\alpha = 0,5$); kalt ≈ 28 s, warm ≈ 5 s je Lösung — deshalb ist Verbier als Batch-/Analysenetz dokumentiert, nicht als Echtzeitnetz.

Kennzahl	rheatflow	gemessen	Delta
Einspeisung HS0+HS1	3567 kW	3590 kW	-0,6 %
HS0 Massenstrom	17,83 kg/s	17,76 kg/s	+0,4 %
HS0 Rücklauf	44,10 °C	43,52 °C	+0,58 K
Netzverluste	297 kW	≈ 317 kW	-6 %

Median 0,83 K über 150 Stationen Der Kern der Validierung sind die 150 Übergabestations-Vorlaufttemperaturen — aus den Eingaben *herausgehalten*: Median $|\Delta T|$ **0,83 K** (Toleranz ≤ 2 K), Mittel 1,29 K, p90 2,68 K (Toleranz ≤ 5 K), Max 12,6 K (Toleranz ≤ 20 K). Ein medianer Absolutfehler von 0,83 K über 150 unabhängig gemessene Stationen, mit den Vorlaufttemperaturen aus den Eingaben gehalten, ist das stärkste Validierungsergebnis der Plattform: Die gekoppelte hydraulisch-thermische Lösung rekonstruiert das reale Temperaturfeld im Median bis auf Messrauschen. Fünf Stationen dominieren die Ausreißerspitze (z. B. S96 mit +12,6 K) — in der `DATASET.md` als Sensor-Mittelung, uniforme 5 °C-Randtemperatur und lokale Fluss-Aufteilung im Netz benannt und analysiert. Der *Median*, nicht das Maximum, ist die Aussage, weil dies reale Messdaten sind.

9.5. Akzeptierte Modellunterschiede

Jedes Band oben ist durch dieselbe kurze Liste bewusster, dokumentierter Entscheidungen geweitet — keine ist ein Fehler, jede ist in der jeweiligen `DATASET.md` begründet: temperaturabhängiges Wasser vs. feste Eigenschaften ($\pm 0,3$ – $0,5$ % auf ΔT -geregelte Größen); quasistatisches Wärmemodell vs. dynamische Rohre (der CE-1-Nachtüberschuss, oben hergeleitet begrenzt); angenommene Randtemperatur, wo die Quelle schweigt (Verbier 5 °C, ± 5 K $\Rightarrow \mp 7$ % Verluste — größer als das -6 %-Delta, das damit *kein* Modellfehler ist); reale Messdaten $\Rightarrow$ bewusst großzügige Toleranzen (der Median zählt, nicht das Maximum).

10. Szenarien

Das Menü **Datei** → **Szenario speichern...** legt den kompletten Aufbau als **Rezept** ab — nicht als Schnappschuss: die Netz-id, die deterministische Lastpolitik, die zur Laufzeit platzierten Anlagen (als Konfiguration, nicht als Zustand), das Op-Protokoll der Abnehmer/Bypässe, die Heizkurven-/ Δp -/Erzeuger-Konfiguration, der Wetter-Override und die Engine-Uhr. Weil jede Zutat deterministisch ist (geseedete Profile, protokollierte Ops), reproduziert das Nachspielen den Aufbau exakt, und die Datei bleibt lesbar — bewusst händeditierbares JSON unter `data/scenarios/`. Das Laden ist *tolerant je Eintrag*: Passt eine Op nach einem Netzwechsel nicht mehr, wird sie mit einer Warnung übersprungen statt mit einem halben 500.

Mitgeliefert sind zwei Szenarien mit **gleichem Seed 42**: `demo-dorf-3g-winter` und `demo-dorf-4g-vergleich`.

Direkter Vergleich der Temperaturniveaus Weil beide Szenarien denselben Seed tragen, sind ihre Lastprofile identisch — der einzige Unterschied ist das Temperaturniveau (und die 4G-Variante fährt eine Erdreich-Wärmepumpe). Lädt man abwechselnd beide, sieht man am selben Netz und denselben Lasten direkt, wie die Verlustquote fällt, der Massenstrom (und Pumpenstrom) steigt und der Wärmepumpen-COP im 4G-Fall besser wird — Kapitel 6.4 und 7.1, aber als greifbarer Seite-an-Seite-Vergleich statt als Tabelle.

11. Daten exportieren

11.1. Aufzeichnung

Datei → Aufzeichnung starten: Jeder veröffentlichte Schritt wandert über eine nicht-blockierende Senke (eigener Schreib-Thread + Queue, die Engine blockiert nie) in ein CSV-Paket unter `data/recordings/<id>/`. Das Paket enthält je eine CSV für `summary`, `junctions`, `pipes`, `consumers`, `producers`, `storages`, `controls` und die Messwerte, dazu eine `metadata.json` als Reproduktions-Rezept. Weil die Senke den *projizierten* Frame konsumiert, gilt der strikte Modus auch für die CSVs: Was nie auf den Draht kommt, kommt nie in die Dateien. Beenden, dann als ZIP herunterladen (**Aufzeichnungen-Flyout**).

11.2. Tage-Export (Bulk)

Datei → Tage exportieren... simuliert ganze Tage des aktuellen Aufbaus offline so schnell wie möglich: Die Engine wird kurz geparkt, der Simulator tief kopiert, auf einen deterministischen Mitternacht-Start normalisiert (`prepare_replay`: Kaltstart, SoC 0, frische Messfenster, geregelte Pumpe auf die Datei-Förderhöhe gelöst) und dann die Tage zurück in ein privates Paket abgespielt. Das Paket ist **byte-kompatibel** zu einer Live-Aufzeichnung.

Was „byte-kompatibel“ bedeutet Ein verkürzter Tag wird einmal live über den Veröffentlichungspfad aufgezeichnet, dann aus einer tiefen Kopie des *driftenden* Simulators (Warmstart fortgeschritten, Δp -Pumpe bewegt, Fenster gefüllt) exportiert — und jede CSV ist *byte-identisch*, nachdem genau zwei dokumentierte volatile Spalten maskiert sind: `timestamp` (Wanduhr) und `solve_ms` (Maschinen-Timing). Das ist der Beweis, dass der Offline-Export dieselbe Physik rechnet wie der Live-Betrieb — nur schneller. Die Schätz-Ebene wird bewusst *nicht* aufgezeichnet (ihre Kadenz ist Maschinen-Timing, kein Physik-Takt; sie ist aus den aufgezeichneten Messwerten neu berechenbar).

11.3. Der experimentelle Transient-Modus

`RTHEATFLOW_TRANSIENT=true` rechnet *nur den Export* mit thermischer Trägheit der Wassersäule (nicht Rohrwand oder Erdreich): Der Solver stellt zwei transiente Stufen (mit explizitem `dt` und monotonem `simulation_time_step`) vor die quasistatische Ladder. Das Verketteten pro Schritt ist genau das, was `run_timeseries` intern tut, und wurde *bit-identisch* nachgewiesen, bevor darauf gebaut wird. Scheitert eine transiente Stufe, fällt der Schritt automatisch auf quasistatisch zurück und das Paket wird markiert (`transient_fallback`). Der *Live*-Betrieb bleibt dagegen *immer* quasistatisch — der Transient-Modus ist ausschließlich der Offline-Export, nie die Echtzeitschleife.

Trägheit sichtbar machen Hebt man im Transient-Export die Vorlauftemperatur der Zentrale an, sieht das ferne Netzeende die neue Temperatur *nicht* sofort: Sie zerfällt über Schritte (358,05 → 357,57 K) statt des quasistatischen sofortigen Sprungs (351,8 K) — die Transportverzögerung, über 1 K, per Test gepinnt. Für sinnvolle Ergebnisse wählt man die Rohr-`sections` so, dass ein Abschnitt etwa $v \cdot dt$ lang ist.

12. Langzeit-Visualisierung mit Grafana

`docker compose up` startet den vollen Visualisierungsstack:

1. **Collector** — pollt `GET /state` alle 0,5 s, entdoppelt auf (`day`, `step`) und schreibt wanduhr-gestempelte Punkte nach InfluxDB: `summary` (alle Kennzahlen + `converged` + Solverstatus + Δp -Soll/Ist + Wetter), `junction`, `pipe`, `consumer`, `producer`, `storage`.
2. **InfluxDB 2.7** — Bucket `heatflow`, Organisation `rheatflow`, 7 Tage Aufbewahrung (Port 8087).
3. **Grafana 11** — vollständig aus dem Repository provisioniert (Datenquelle + Dashboard-JSON), Port 3001, Zugang `admin/admin`.

Die mitgelieferten Panels des Fernwärme-Dashboards: Vorlauf/Rücklauf der Zentrale gegen den Heizkurven-Sollwert; Schlechtpunkt- Δp gegen Sollwert und Förderhöhe (mit 0,5-bar-Schwellenlinie); Erzeuger-Dispatch als Stapel; Speicher-SoC; Verlustquote als Anzeige; Rechenzeit als Statwert; und der Solverstatus als OK/DEGRADED/FAILED-Abbildung. Sieht Grafana nichts, wartet der Collector, bis `rheatflow` und InfluxDB gesund sind und der erste Schritt gelöst ist — Zeitbereich auf „letzte 5 Minuten“ stellen.

13. Konfiguration

Alle Einstellungen erfolgen über Umgebungsvariablen mit dem Präfix `RTHEATFLOW_` (eine `.env`-Datei wird gelesen; Vorlage: `.env.example`). In der Tabelle ist das Präfix weggelassen.

Variable	Vorgabe	Bedeutung
<code>DATA_DIR</code>	<code>./data</code>	Datenverzeichnis
<code>DEFAULT_NETWORK</code>	<code>demo_dorf</code>	Netz beim Start
<code>STEP_INTERVAL_SECONDS</code>	<code>1.0</code>	echte Sekunden je Simulationsminute (0,01...)
<code>STEPS_PER_DAY</code>	<code>1440</code>	Schritte pro Tag
<code>AUTOSTART</code>	<code>true</code>	Simulation beim Start anlaufen lassen
<code>HISTORY_SIZE</code>	<code>1440</code>	Größe des Verlaufspuffers
<code>SOLVER_ITER</code>	<code>100</code>	Basis-Iterationen der Retry-Ladder
<code>PUMP_ETA</code>	<code>0.7</code>	elektr. Pumpenwirkungsgrad ($P_{el} = P_{hyd}/\eta$)
<code>DP_MIN_BAR</code>	<code>0.5</code>	Mindest- Δp am kritischen Abnehmer [bar]
<code>RETURN_TEMP_MARGIN_K</code>	<code>5</code>	Bernstein-Schwelle Rücklaufverletzung [K]
<code>MIN_QEXT_W</code>	<code>500</code>	Nullfluss-Schutz: Abnehmer-Lastuntergrenze [W]
<code>EXPOSE_GROUND_TRUTH</code>	<code>true</code>	<code>false</code> = strikter Modus (Realität verlässt den Server nicht)
<code>TRANSIENT</code>	<code>false</code>	<code>true</code> = Offline-Export mit thermischer Trägheit (experimentell)
<code>RECORD</code>	<code>false</code>	<code>true</code> = Daueraufzeichnung, ein Paket je Konfiguration
<code>HOST / PORT</code>	<code>127.0.0.1 / 8001</code>	API-Bindung (Docker/LAN: 0.0.0.0); Geschwisterschema
<code>CORS_ORIGINS</code>	<code>*</code>	erlaubte Browser-Origins
<code>LOG_LEVEL</code>	<code>info</code>	Log-Ausführlichkeit

.env für eine Prüfungsumgebung Doppeltes Tempo, strikte Beobachtbarkeit (Studierende können die Wahrheit auch per API nicht abfragen), Zugriff aus dem Kursnetz:

```
RTHEATFLOW_STEP_INTERVAL_SECONDS=0.5
RTHEATFLOW_EXPOSE_GROUND_TRUTH=false
RTHEATFLOW_HOST=0.0.0.0
```


A. REST-/WebSocket-Schnittstelle

Für Skripte, eigene Frontends oder Übungsaufgaben ist die komplette Funktionalität auch per HTTP erreichbar (Standard: <http://127.0.0.1:8001>, ohne Authentifizierung — Lehrwerkzeug). Die Plattform hat **61 Routen**; die *vollständige* Referenz steht in `docs/API.md` (aus dem Code generiert) und interaktiv in Swagger unter `/docs`. Auswahl der wichtigsten Kategorien:

Methode	Pfad	Beschreibung
GET	<code>/health</code> , <code>/status</code>	Diagnose, Engine-Zustand
GET	<code>/network</code>	statische Topologie (inkl. Geokoordinaten)
GET	<code>/state</code>	letzter gelöster Schritt (404 vor der ersten Lösung)
GET	<code>/history?limit=N</code>	Verlaufspuffer
WS	<code>/ws</code>	ein projizierter <code>StepResult</code> je Schritt
GET	<code>/manual</code>	dieses Handbuch (HTML; <code>?format=mdroh</code>)
POST	<code>/control/start pause resume</code>	Simulation steuern
POST	<code>/control/seek seekday interval</code>	Zeit/Tag/Schrittdauer setzen
GET/PUT/DEL	<code>/weather...</code>	Wetter + Außentemperatur-Override
GET/POST	<code>/heatingcurve</code> , <code>/dpcontrol</code>	Heizkurve, Schlechtpunktregelung
GET/POST/DEL	<code>/producer...</code> , <code>/storage...</code>	Erzeuger, Speicher
POST/DEL	<code>/consumer...</code> , <code>/bypass</code>	Abnehmer, Bypass
GET/POST/DEL	<code>/measurements...</code>	Messstellen, Presets, Modus
GET/POST	<code>/estimation/config</code>	Beobachter-Richtlinie
GET/POST	<code>/networks...</code> , <code>/config/apply</code>	Katalog, Import, Netz laden
POST	<code>/loadgen/assign</code>	Lastpolitik-Vorschau
GET/POST/DEL	<code>/scenarios...</code>	Szenarien speichern/laden/löschen
GET/POST	<code>/recording...</code> , <code>/export...</code>	Aufzeichnung, Bulk-Export

Fehlercode-Konventionen: 400 Validierung/Import, 404 fehlende Ressource (oder `/state` vor der ersten Lösung), 409 Konflikte (zweiter Druck-Slack, laufender Export), 422 semantische Grenzen (Wetter-Override außerhalb $-30 \dots 45^\circ\text{C}$), 500 nur interne Fehler — **Solver-Nichtkonvergenz ist Daten (`converged=false-Frames`), nie ein 500.**

A.1. Das StepResult-Wireformat

Das einzige Wireformat ist der projizierte `StepResult`: `/state`, die `/history`-Einträge und jede `/ws`-Nachricht teilen einen `asdict()`+Projektions-Pfad. Im strikten Modus fehlen die vier Wahrheits-Schlüssel (`junctions`, `pipes`, `consumers`, `summary`), `measurements/observed_summary` und `estimated` bleiben.

```
$ curl http://127.0.0.1:8001/state
{
  "step": 440, "day": 0, "time_of_day": "07:20",
  "converged": true, "solver_status": "ok", "solve_ms": 24.1,
  "junctions": [...], "pipes": [...], "consumers": [...],
  "summary": { "q_feed_kw": ..., "q_loss_kw": ..., "loss_pct":
    ...,
    "dp_worst_bar": ..., "worst_consumer": "...",
    "t_flow_plant_c": ..., "balance_err_kw": ... },
  "producers": [...], "storages": [...],
  "weather": {...}, "controls": { "dp_control": { "blind_spot":
    false, ... } },
  "measurements": { "consumers": [...], "nodes": [...], "plant":
    {...} },
  "observed_summary": { "dp_worst_bar": ..., "n_metered": ... },
  "estimated": { "summary": {...}, "error": { "max_dt_return_k":
    ... } }
}
```

Drei typische Aufrufe Ein Netz mit Lastpolitik laden (entspricht **Anwenden**):

```
curl -X POST http://127.0.0.1:8001/config/apply \
-H "Content-Type: application/json" \
-d '{"network_id": "schutterwald",
  "loadgen": {"temperature_preset": "3G", "seed": 42}}'
```

Einen Wärmemengenzähler an Abnehmer 5 setzen und den Rücklauf abfragen:

```
curl -X POST http://127.0.0.1:8001/measurements/consumer/5
```

Den Live-Stream anzapfen — jede Nachricht ist ein `JSON-StepResult`:

```
async with websockets.connect("ws://127.0.0.1:8001/ws") as ws:
    result = json.loads(await ws.recv())
```

B. Eingabedateiformat

Ein Netz besteht aus **fünf** DH-nahen JSON-Dateien. Der Vertrag ist *einseitig*: Die Topologie-Dateien beschreiben *Trassenknoten*, nicht das Vor-/Rücklauf-Paar, das der Solver simuliert — der Netzbauer entfaltet das Paar (Kapitel 6), handgeschriebene Dateien bleiben halb so groß. Jedes Modell verbietet unbekannte Schlüssel (`extra="forbid"`), Tippfehler fallen also hart durch.

network_structure.json Trassenknoten (`name`, `kind`, `geo` als `[lat, lon]`, `pn_bar`).

Der Bauer erzeugt aus jedem Eintrag das `_s/_r`-Junction-Paar.

pipes.json Trassen (`from_node`, `to_node`, `length_km`, ISOPLUS-`std_type` oder explizite `inner_diameter_mm/u_w_per_m2k`, optional `geometry`). Std-Typ und Einzelparameter schließen sich aus.

consumers.json die Profil-Zeilen *sind* die Übergabestationen: `q_sh_w/q_dhw_w` (Raum/Warmwasser) plus genau ein Partner: `treturn_k` oder `deltat_k` oder `controlled_mdot_kg_per_s`.

producers.json genau ein Druck-Slack (`circ_pump_const_pressure`, mit optionaler `heating_curve` und `dp_control`); Sekundäre als `heat_exchanger` oder `pump_mass`.

weather.json `t_amb_c` + `t_ground_c`, je Länge `steps`.

Beim Laden akkumulieren alle Kreuz-Prüfungen (Knotenverweise, Array-Längen, Horizont-Ausrichtung, genau-ein-Slack, Erreichbarkeit vom Slack, Nullfluss/Sackgassen) in *eine* Fehlermeldung. Die vollständige Vertragsbeschreibung und die Konvertierungsentscheidungen der Referenznetze stehen in `docs/REFERENCE_NETWORKS.md`.

Minimalnetz: Zentrale, eine Trasse, ein Haus

```
// network_structure.json
{ "name": "Minimal",
  "junctions": [
    { "name": "Zentrale", "kind": "plant",    "geo": [48.0,
      7.8], "pn_bar": 6.0 },
    { "name": "Haus",    "kind": "consumer", "geo": [48.0,
      7.81], "pn_bar": 6.0 } ] }

// pipes.json
{ "pipes": [ { "from_node": "Zentrale", "to_node": "Haus",
  "length_km": 0.2, "std_type": "ISOPLUS_DRE50_STD"
  } ] }
```

```
// consumers.json (steps Werte je Profil; hier angedeutet)
{ "resolution_minutes": 1, "steps": 1440,
  "consumers": [ { "node": "Haus", "q_sh_w": [8000, ...],
                  "q_dhw_w": [1000, ...], "treturn_k": [328.15,
                  ...],
                  "q_design_w": 12000 } ] }

// producers.json (der Druck-Slack)
{ "producers": [ { "kind": "slack", "node": "Zentrale",
                  "p_flow_bar": 6.0, "plift_bar": 2.0, "
                  t_flow_k": 358.15 } ] }

// weather.json
{ "resolution_minutes": 1, "steps": 1440,
  "t_amb_c": [2.0, ...], "t_ground_c": [8.0, ...] }
```

C. Fehlerbehebung

Port 8001 oder 5174 belegt Der Starter erkennt einen bereits laufenden rheatflow und startet ihn nicht doppelt. Eine fremde Anwendung auf dem Port lässt sich mit `stop_rheatflow.bat` räumen — es beendet die Serverfenster, die Besitzer der Ports 8001/5174 und verwaiste node-/esbuild-/python-Prozesse dieses Repos (erkannt über die *Kommandozeile*, nicht den Exe-Pfad). Ein parallel laufendes netzsim/rtpowerflow (8000/5173) bleibt unberührt — das ist der Sinn des Geschwister-Portschemas.

Der Start hängt vor dem Binden Meist der numba-Kaltstart (Abschnitt 2.4): pandapi-pes/numba importieren und der erste JIT-Lauf dauern zusammen mehrere Sekunden bis knapp eine Minute. Ein echter Portkonflikt ist dagegen ein *schneller WinError* 10048 mit sofortigem Prozessende, kein Hängen.

/state liefert 404 Normal direkt nach dem Start — der erste Schritt ist noch nicht gelöst. Nach spätestens einer Schrittdauer (plus Kaltstart) verschwindet der Fehler von selbst.

Solver „degradiert“ oder „gescheitert“ Die Kopfzeile vermerkt den Status; die Simulation heilt sich im nächsten Schritt selbst (Nichtkonvergenz *ist* Daten, kein Absturz). „degradiert“ heißt: Stufe 4 der Retry-Ladder (sequentiell) hat gelöst, aber die Sollwerte sind nicht gehalten. Häufigste Ursache für „gescheitert“: eine extreme Lastpolitik oder ein stark vermaschtes Netz — didaktisch oft genau der Punkt, an dem das Netz physikalisch am Ende ist. Verbier braucht regulär Tier 2.

Die Karte liegt im Wasser War ein Anker-Bug und ist behoben: DESTEST und Verbier tragen nur lokale Meter ohne CRS, ihre Geokoordinaten sind synthetische Anker. Frühere Versionen ankerten über offenem Wasser (Bodensee/Lac Léman); jetzt liegen die Anker an Land. Wer eigene Netze aus quellenlosen Metern konvertiert, ankert sie ebenfalls an Land.

Skriptzugriff auf den Vite-Dev-Server scheitert Der Vite-Server bindet unter Windows nur IPv6 (:::1). Browser sind fein (localhost löst zuerst nach :::1 auf), aber ein IPv4-127.0.0.1-curl gegen die Dev-UI wird abgewiesen — für Skripte `http://[:::1]:5174` verwenden. (Das *Backend* auf 8001 ist über IPv4 erreichbar.)

Grafana zeigt nichts Der Collector wartet, bis rheatflow und InfluxDB gesund sind und der erste Schritt gelöst ist; Zeitbereich auf „letzte 5 Minuten“ stellen.

Die Schätzung erscheint nicht oder springt Ist der Beobachter deaktiviert (POST `/estimation/config` mit `enabled=false`), fehlt die `estimated`-Ebene; die Sicht fällt dann auf Realität (bzw. im strikten Modus auf Gemessen) zurück. Auf großen Netzen zieht die Schätzung wegen der Selbstdrosselung in Sprüngen nach — sichtbare Rechenzeit, kein Fehler (Abschnitt 5.3).